

UNIVERSITÉ DE LIEGE



RAPPORT PROJET

INFO0054 - Programmation Fonctionnelle

Projet : Tableaux Sémantiques

Samuel DAOUD (s181356)
Sam EL MASRI (s190377)
Gregory VOSKERTCHIAN (s190733)

2021-2022

1 Extensions

1. Les règles d'éliminations pour les opérateurs EQUIV, NAND, XOR et XNOR sont détaillées au sein de cette section.

ϕ	π	$\phi \equiv \pi$
0	0	1
0	1	0
1	0	0
1	1	1

TABLE 1 – EQUIV truth table

ϕ	π	$\neg(\phi \wedge \pi)$
0	0	1
0	1	1
1	0	1
1	1	0

TABLE 2 – NAND truth table

ϕ	π	$\phi \oplus \pi$
0	0	0
0	1	1
1	0	1
1	1	0

TABLE 3 – XOR truth table

- **EQUIV** : La table 1 nous donne la table de vérité. Nous pouvons voir que $\phi \equiv \pi$ est vrai si :

$$(\phi \text{ is } False \text{ AND } \pi \text{ is } False) \text{ OR } (\phi \text{ is } True \text{ AND } \pi \text{ is } True)$$

- **NAND** : La table 2 nous donne la table de vérité. Nous pouvons voir que $\neg(\phi \wedge \pi)$ est vrai si :

$$(\phi \text{ is } False \text{ OR } \pi \text{ is } False)^1$$

- **XOR** : La table 3 nous donne la table de vérité. Nous pouvons voir que $\phi \oplus \pi$ est vrai si :

$$(\phi \text{ is } False \text{ AND } \pi \text{ is } True) \text{ OR } (\phi \text{ is } True \text{ AND } \pi \text{ is } False)$$

- **XNOR** : La table 2 nous donne la table de vérité. Nous pouvons voir que $\neg(\phi \oplus \pi)$ est vrai si :

$$(\phi \text{ is } False \text{ AND } \pi \text{ is } False) \text{ OR } (\phi \text{ is } True \text{ AND } \pi \text{ is } True)$$

1. Cette règle fut en réalité déjà donnée dans la présentation

ϕ	π	$\neg(\phi \oplus \pi)$
0	0	1
0	1	0
1	0	0
1	1	1

TABLE 4 – XNOR truth table

2. Étendre les règles d'éliminations pour le OR et le AND prennant plusieurs arguments est direct

$(OR\ n_1\ n_2\ n_3\ \dots)$ est *Vrai* si **au moins une** des n propositions est *Vrai*.
Donc si n_1 OU n_2 OU n_3 OU ... est *Vrai*.

$(AND\ n_1\ n_2\ n_3\ \dots)$ is *Vrai* si **toutes** les n propositions sont *Vrai*.
Donc si n_1 ET n_2 ET n_3 ET ... sont *Vrai*.

La figure 1 ci-dessous illustre les résultats obtenus lorsque ces opérateurs sont appliqués à 3 arguments.

x	y	z	$AND(x,y,z)$	$OR(x,y,z)$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

FIGURE 1 – Table de vérité de AND et de OR avec 3 arguments

2 Formules et tableaux

Une formule sera simplement représentée par un arbre ou chaque liste sera une opération booléenne. Les feuilles (les éléments) seront soit une proposition soit un opérateur.

Ex : $((OR\ p\ q)\ r\ (NOT\ q))$

Un tableau est représenté par une liste contenant soit une proposition, donc un élément, soit une opération NOT appliquée à une seule proposition, donc une liste de deux éléments.

La fonction *semtab* renvoie une liste de tableaux.

Ex : $((p\ r\ q)\ (q\ r\ (NOT\ q)))$

3 Implémentation des règles d'éliminations

Lors de notre implémentation de la fonction *semtab*, nous avons été contraint à utiliser le concept des fonctions auxilliaires. En effet, afin de pouvoir retenir les résultats intermédiaires au cours de notre élimination d'opérateur nous avons besoin de stocker la partie de la liste de l'ensemble de formules déjà traitées (dans le code cette variable = *previous*) afin de prévoir l'apparition d'un

opérateur (comme par exemple si on a une formule $p \wedge q$) qui nous contraint à diviser le tableau en deux (dans le cas de notre exemple, on obtient : $((p) (q))$). Donc, il faut stocker la liste de formules déjà traité pour pouvoir la copier dans deux nouveaux tableau pour ensuite continuer notre élimination d'opérateurs de formules de notre liste de formules initiale.

L'implémentation de notre code contient donc une fonction `semtab`, qui fait appelle à une fonction auxilliare nommée `semtab-aux`, à laquelle nous donnons en argument la liste de formule qu'il reste à traiter et la liste de formules déjà traitée (variable auxilliare). Donc pour notre première appelle à la fonction auxilliare, il nous faut donner en argument la liste de formules initiale `f` et la liste nulle `'()` car rien n'a encore été éliminé.

Dans cette fonction, nous faisons les vérifications requise afin de détecter pour chaque formule de la liste les différents opérateurs auxquels une élimination est possible. Puis si un opérateur précis est détecté alors on fait appel à des sous-fonctions auxiliaires pour faire les opérations requises d'éliminations d'opérateurs. Nous avons décidé d'utiliser ces sous-fonctions auxiliaires afin que le code soit plus lisible et compréhensible. Puis ces sous-fonctions refont appels à `semtab-aux`, et que ce processus continue jusqu'à ce que la liste qu'il reste à traitée soit nulle, nous renvoyons au final notre variable auxiliaire previous qui contient un tableau de formules sans les opérateurs à éliminer. Si il y a eu division de "tableaux" (c'est à dire que nous avons eu à faire à un OR par exemple) et que nous nous retrouvons avec plus de 1 tableau, alors ceux-ci vont se recoller pour former une liste de tableaux. Cette dernière opération peut être réalisée grâce à la fonction `append-map`.

Cette fonction à la propriété de concaténer des listes (dans notre cas des tableaux) en prenant en argument seulement la liste de formules pour lesquelles l'opération spécifier dans les lambdas doit être réalisée. La liste donnée en argument peut donc contenir un nombre "illimité" de formules. Cette dernière propriété nous permet donc de déjà avoir une règle d'élimination valide pour un OR prenant un nombre d'arguments supérieur à 2. Chose qui nous est demandée dans l'extension.

4 Implémentation de l'extension

Notre première implémentation de `semtab` permettait déjà l'utilisation de plus de deux arguments pour les opérateurs AND et OR.

La règle d'élimination de l'extension *NAND* est en soit déjà présente au sein de `semtab` puisqu'en effet cet opérateur correspond au *NOT AND* déjà considéré. Toutefois, afin que cette notation puisse être utilisé dans une formule, un cas *NAND* a été ajouté. Cependant si un utilisateur souhaitait utilisé cet opérateur il ne pourrait le faire qu'avec deux arguments tandis que la notation *NOT AND* permet de le faire avec davantage.

Les opérateur *EQUIV* et *XNOR* possèdent la même règle d'élimination. Les deux cas sont tout de mêmes ajoutés à `semtab` afin de permettre, de nouveau, l'utilisation des deux notations dans une formule.

5 Modules et utilisations

Pour illustrer l'utilisation de nos fonctions nous allons considérer la formule suivante $p \wedge q, r, \neg q$ que l'on notera `f`.

Cette formule sera implémentée avec la notation $'((\text{OR } p \text{ } q) \text{ } r \text{ } (\text{NOT } q))$.

L'application de nos fonctions à **f** donne les résultats :

$(\text{semtab } f) \implies '((p \text{ } r \text{ } (\text{NOT } q)) \text{ } (q \text{ } r \text{ } (\text{NOT } q)))$

$(\text{satisfiable? } f) \implies \#t$

$(\text{tautology? } f) \implies \#f$

$(\text{contradiction? } f) \implies \#f$

$(\text{models } f) \implies '((p \text{ } r \text{ } (\text{NOT } q)))$

Les résultats renvoyés par les fonctions *valid?* et *counterexamples* ne sont pas listés car nous n'avons malheureusement pas été capable de les implémenter.